

Prometheus + Grafana: Production Monitoring Guide

A step-by-step reusable workflow for building production monitoring for AI/LLM systems and other services.

1. The Complete Workflow

1. Decide what to monitor
- ↓
2. Create metrics in Python
- ↓
3. Update metrics when events happen
- ↓
4. Expose /metrics endpoint
- ↓
5. Configure Prometheus
- ↓
6. Verify Prometheus scraping
- ↓
7. Write PromQL queries
- ↓
8. Connect Grafana → Prometheus
- ↓
9. Create dashboard
- ↓
10. Create panels/charts
- ↓
11. Create alerts
- ↓
12. Test with simulated incidents

2. Decide What You Want to Monitor

Before writing Prometheus code, define the signals you actually need.

System

- requests
- errors
- latency
- throughput

LLM

- input tokens
- output tokens
- latency
- failures

RAG

- retrieval latency
- chunks retrieved
- similarity

Quality

- faithfulness
- relevance

Security

- PII
- prompt injection
- jailbreak

Rule: Don't start with Prometheus code. Start with the metrics you actually need.

3. Install the Prometheus Python Client

```
pip install prometheus-client
```

4. Import Prometheus Metric Classes

```
from prometheus_client import Counter, Gauge, Histogram
```

5. Understand the Three Important Metric Types

Counter

Use Counter when something only goes upward: total requests, errors, tokens, or violations.

```
REQUESTS_TOTAL = Counter(  
    "ai_requests_total",  
    "Total number of AI requests"  
)  
  
REQUESTS_TOTAL.inc()  
REQUESTS_TOTAL.inc(10)
```

Remember: Counter = How many times did something happen?

Gauge

Use Gauge for values that can go up and down: CPU, memory, queue size, or the latest quality score.

```
FAITHFULNESS_SCORE = Gauge(  
    "ai_faithfulness_score",  
    "Latest simulated faithfulness score"  
)  
  
FAITHFULNESS_SCORE.set(0.91)  
FAITHFULNESS_SCORE.set(0.76)
```

Remember: Gauge = What is the value right now?

Histogram

Use Histogram for distributions such as latency, request duration, response size, and retrieval latency.

```
REQUEST_LATENCY = Histogram(  
    "ai_request_latency_seconds",  
    "AI request latency in seconds",  
    buckets=[0.5, 1, 2, 3, 5, 10, 20]  
)  
  
REQUEST_LATENCY.observe(2.4)
```

Remember: Histogram = How are values distributed? It supports P50, P95, and P99 analysis.

6. Create Your Metrics Module

```
app/  
├── monitoring/  
│   ├── __init__.py  
│   ├── prometheus_metrics.py  
│   └── ...  
  
from prometheus_client import Counter, Histogram, Gauge  
  
REQUESTS_TOTAL = Counter(  
    "ai_requests_total",  
    "Total number of AI requests"  
)
```

```

ERRORS_TOTAL = Counter(
    "ai_errors_total",
    "Total number of AI request errors"
)

REQUEST_LATENCY = Histogram(
    "ai_request_latency_seconds",
    "AI request latency in seconds",
    buckets=[0.5, 1, 2, 3, 5, 10, 20]
)

FAITHFULNESS_SCORE = Gauge(
    "ai_faithfulness_score",
    "Latest faithfulness score"
)

```

7. Record Metrics When an Event Happens

```

def record_event(event):

    REQUESTS_TOTAL.inc()

    if event["status"] == "error":
        ERRORS_TOTAL.inc()

    REQUEST_LATENCY.observe(
        event["latency_ms"] / 1000
    )

    FAITHFULNESS_SCORE.set(
        event["faithfulness"]
    )

```

Production pattern: Application event → record_event() → Prometheus metric updated.

8. Expose Metrics

Prometheus cannot see Python variables directly. Your application or exporter must expose a /metrics endpoint.

```

from prometheus_client import start_http_server

start_http_server(8001)

```

Open <http://localhost:8001/metrics> and verify that Prometheus-formatted metrics are visible.

```

# HELP ai_requests_total Total number of AI requests
# TYPE ai_requests_total counter
ai_requests_total 1250

# HELP ai_faithfulness_score Latest simulated faithfulness score
# TYPE ai_faithfulness_score gauge
ai_faithfulness_score 0.91

```

9. Start Generating Metrics

```

while True:

    event = generate_request()

    record_event(event)

    time.sleep(1)

```

Dummy event → record_event() → Counter/Gauge/Histogram → /metrics.

10. Configure Prometheus

```
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: "ai-monitoring"

    static_configs:
      - targets:
        - "localhost:8001"
```

scrape_interval: 5s means Prometheus asks the exporter for metrics every 5 seconds. The target tells Prometheus where to scrape.

11. Start Prometheus

```
.\prometheus.exe --config.file=prometheus.yml
```

Open <http://localhost:9090>.

12. Verify Prometheus Is Scraping

Go to Status → Targets. You want to see the monitoring target as UP.

```
ai-monitoring
localhost:8001
UP
```

If it is DOWN, check the exporter, port 8001, /metrics, and prometheus.yml.

13. Test Your First PromQL Queries

```
ai_requests_total
ai_errors_total
ai_faithfulness_score
```

14. Convert Counters Into Rates

```
rate(ai_requests_total[1m])
```

This gives approximately requests per second over the last minute.

15. Calculate Error Rate

```
(
  rate(ai_errors_total[1m])
  /
  rate(ai_requests_total[1m])
) * 100
```

16. Calculate P95 Latency

```
histogram_quantile(
  0.95,
  rate(ai_request_latency_seconds_bucket[5m])
)
```

If the result is 4.2, interpret it as approximately 4.2 seconds P95 latency.

17. Calculate P99 Latency

```
histogram_quantile(
  0.99,
```

```
    rate(ai_request_latency_seconds_bucket[5m])
  )
```

18. Monitor Token Usage

```
rate(ai_input_tokens_total[5m])
rate(ai_output_tokens_total[5m])
```

19. Monitor RAG Retrieval Latency

```
histogram_quantile(
  0.95,
  rate(ai_retrieval_latency_seconds_bucket[5m])
)
```

20. Monitor AI Quality

```
ai_faithfulness_score
ai_answer_relevance_score
```

Infrastructure can be healthy while AI quality is poor, so AI quality signals belong in production monitoring.

21. Monitor Security

```
rate(ai_prompt_injection_total[5m])
rate(ai_pii_violations_total[5m])
rate(ai_jailbreak_total[5m])
```

22. Connect Grafana to Prometheus

Start Grafana and open <http://localhost:3000>.

```
Connections
  ↓
Data sources
  ↓
Add data source
  ↓
Prometheus
```

Set the Prometheus URL to <http://localhost:9090> and use Save & Test.

23. Create a Grafana Dashboard

```
Dashboards
  ↓
New
  ↓
New dashboard
  ↓
Add visualization
```

Select Prometheus as the data source. Mental model: Prometheus → PromQL → Grafana panel → visualization.

24. Create Request Rate Chart

```
rate(ai_requests_total[1m])
```

Use a Time series visualization. This shows request traffic over time.

25. Create Error Rate Chart

```
(
  rate(ai_errors_total[1m])
  /
  rate(ai_requests_total[1m])
) * 100
```

Use Time series, set unit to Percent (0-100), and title it Error Rate.

26. Create P95 Latency Chart

```
histogram_quantile(
    0.95,
    rate(ai_request_latency_seconds_bucket[5m])
)
```

Use Time series, set unit to seconds, and title it P95 Request Latency.

27. Create P99 Latency Chart

```
histogram_quantile(
  0.99,
  rate(ai_request_latency_seconds_bucket[5m])
)
```

Use Time series and title it P99 Request Latency.

28. Create Current Quality Score

ai_faithfulness_score

A Stat panel is useful for a current quality score. Repeat for ai answer relevance score.

29. Create Security Panels

```
rate(ai_prompt_injection_total[5m])
rate(ai_pii_violations_total[5m])
rate(ai_jailbreak_total[5m])
```

Use Time series for trends and Stat when you primarily want the current value.

30. Recommended Grafana Dashboard Layout

[illegible]

Organize dashboards by purpose instead of adding random charts.

31. Add Prometheus Alert Rules

```

- alert: HighErrorRate

  expr: |
    (
      rate(ai_errors_total[1m])
      /
      rate(ai_requests_total[1m])
    ) > 0.05

  for: 30s

  labels:
    severity: critical

  annotations:
    summary: "AI service error rate is high"

```

Pattern: Metric → PromQL expression → threshold → duration → alert.

32. Why Use a 'for' Duration?

A brief one-second spike may not be a real incident. A condition that remains above the threshold for 30 seconds is more meaningful and reduces alert noise.

33. Test Your Alert

```

python -m app.monitoring.metrics_exporter normal
python -m app.monitoring.metrics_exporter latency_spike
python -m app.monitoring.metrics_exporter rag_degradation
python -m app.monitoring.metrics_exporter security_attack

```

Verify that the metric changes, the Grafana dashboard changes, and the expected alert fires.

34. Complete Production Debugging Flow

Someone reports: "The AI application is slow."

```

Grafana
↓
Is P95/P99 high?
↓
YES
↓
Which component?
↓
Trace
↓
LLM / RAG / DB / API
↓
Logs
↓
Root cause

```

Example: P95 increases → LLM latency increases → input tokens increase → context size increases → retriever returns too much data → RAG configuration problem.

35. Prometheus Cheat Sheet

Counter

```

Counter(...)
counter.inc()
rate(metric_total[5m])

```

Use for requests, errors, tokens, and violations.

Gauge

```
Gauge(...)
gauge.set(value)
metric_name
```

Use for current scores, CPU, memory, and queue size.

Histogram

```
Histogram(...)
histogram.observe(value)

histogram_quantile(
    0.95,
    rate(metric_bucket[5m])
)
```

Use for latency, request duration, and response-size distributions.

36. Ten PromQL Queries to Know

```
ai_requests_total

rate(ai_requests_total[1m])

rate(ai_errors_total[1m])

(
    rate(ai_errors_total[1m])
    /
    rate(ai_requests_total[1m])
) * 100

histogram_quantile(
    0.95,
    rate(ai_request_latency_seconds_bucket[5m])
)

histogram_quantile(
    0.99,
    rate(ai_request_latency_seconds_bucket[5m])
)

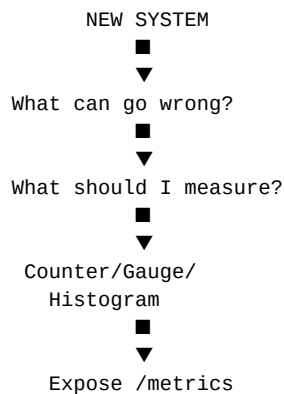
rate(ai_input_tokens_total[5m])

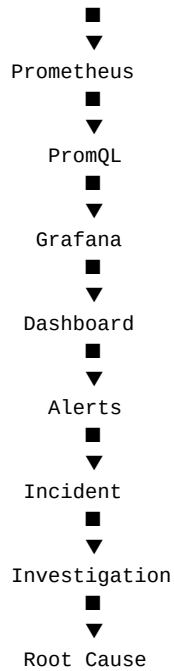
rate(ai_output_tokens_total[5m])

ai_faithfulness_score

rate(ai_prompt_injection_total[5m])
```

37. Final Mental Model





38. The One Sentence to Remember

Prometheus collects and stores metrics; PromQL analyzes those metrics; Grafana visualizes them; alerting turns abnormal behavior into action.

For an AI Engineer, also monitor LLM latency/cost, RAG behavior, AI quality, and safety/security signals.

39. Generic Production Monitoring Checklist

- [] Understand system architecture
- [] Define healthy behavior
- [] Define SLOs / thresholds
- [] Decide which signals matter
- [] Add Counters
- [] Add Gauges
- [] Add Histograms
- [] Expose /metrics
- [] Configure Prometheus
- [] Verify targets are UP
- [] Write PromQL
- [] Connect Grafana
- [] Build dashboards
- [] Add alert rules
- [] Define alert severity
- [] Connect alerts to incidents
- [] Investigate with metrics + logs + traces
- [] Perform root-cause analysis
- [] Test monitoring with simulated incidents
- [] Monitor the monitoring system
- [] Continuously improve